Beijing Institute of Technology

School of Computer Science and Technology

Export Trade Data Analyzer

Big Data Insights for International Commerce

A Hadoop MapReduce Application Processing 600,000 Real Trade Records

Course: Big Data Processing Technology

Instructor: Prof. Meihui Zhang

Maximo Fox **Tomas Cruz**

Spring 2026

Table of Contents

1. Introduction **3**
1.1 Background 3
1.2 Motivation 3
1.3 Objectives 3
2. Group Members and Task Assignments **4**
3. System Design **5**
3.1 Architecture Overview 5
3.2 MapReduce Job Design 5
4. Platform and Tools **6**
5. Implementation **7**
5.1 Data Acquisition 7
5.2 Data Cleaning 8
5.3 Hadoop Cluster Setup and HDFS Ingestion 9
5.4 MapReduce Job Execution 10
6. Results and Demonstration **11**
6.1 Job 1: Export Volume by Country 11
6.2 Job 2: Top Products per Market 11
6.3 Job 4: Emerging Markets 12
6.4 Job 5: Yearly Evolution 12
6.5 Interactive Dashboard 13
7. Difficulties and Solutions **15**
7.1 NameNode Failure on Startup 15
7.2 CSV Parsing with Embedded Commas 16
7.3 API Rate Limiting and Server Errors 16
7.4 Dashboard CDN Compatibility 16
8. Conclusion **17**
8.1 Summary of Achievements 17
8.2 Lessons Learned 17
8.3 Future Work 17
References **18**
Appendix **19**

1. Introduction

1.1 Background

International trade is one of the most data-intensive domains in the global economy. Every year, customs agencies, shipping companies, and trade organizations generate terabytes of transaction records documenting the flow of goods between nations. China, as the world's largest exporter, produces an especially vast volume of trade data, with hundreds of thousands of distinct product categories shipped to over 200 destination countries annually.

Understanding these trade patterns is critical for exporters, policymakers, and market analysts. However, the sheer scale of the data presents a fundamental challenge: traditional analytical tools such as spreadsheets and single-server databases cannot efficiently process millions of trade records across multiple dimensions simultaneously. An exporter seeking to identify which products are trending in which markets, how prices are evolving over time, and which countries represent emerging opportunities faces a computationally intensive task that demands a distributed computing approach.

1.2 Motivation

This project was motivated by a real-world need: providing actionable intelligence to exporters through large-scale data analysis. The United Nations Comtrade database, which is the most comprehensive source of international trade statistics, contains millions of records that are publicly available but impractical to analyze without Big Data infrastructure. Our goal was to bridge the gap between raw data availability and actionable business insights.

1.3 Objectives

The project set out to achieve the following analytical objectives:

- Rank all export destination countries by total trade value to identify China's most important markets.
- Determine the top exported product categories for each destination market using HS commodity codes.
- Analyze unit price trends over time to detect inflation, deflation, and volatility in specific product categories.
- Calculate year-over-year export growth rates per country to identify emerging markets with the highest demand acceleration.
- Track the evolution of export volumes and product composition across years to reveal structural shifts in China's trade profile.

All objectives were achieved through a pipeline of six MapReduce jobs executed on Apache Hadoop, processing 600,000 real trade records obtained from the UN Comtrade API.

2. Group Members and Task Assignments

The project was carried out by a team of two members. The table below details the responsibilities assigned to each member throughout the development process.

Member	Role	Responsibilities
Maximo Fox	System Architect & Data Engineer	System architecture design, Hadoop cluster setup and HDFS configuration, data acquisition via UN Comtrade API, MapReduce job design and implementation (Jobs 1, 2, 4), results post-processing pipeline (JSON generation)
Tomas Cruz	Lead Developer & Visualization	Data cleaning pipeline development, MapReduce job implementation (Jobs 3, 5, 6), interactive web dashboard design and development with Chart.js, Vercel deployment, project report writing, presentation preparation, and results interpretation and analysis

3. System Design

3.1 Architecture Overview

The system follows a four-stage pipeline architecture designed to transform raw trade data into interactive visualizations. Each stage is modular and independently testable, which allowed for iterative development and debugging.

Stage 1 — Data Acquisition and Cleaning: Raw trade data is obtained from the UN Comtrade API using a Python script that programmatically downloads 100,000 records per API call across six years (2018-2023). A separate data cleaning script validates, normalizes, and filters the records before ingestion. This stage runs outside Hadoop on the local filesystem.

Stage 2 — HDFS Ingestion: The cleaned CSV file (96.4 MB, approximately 586,000 records) is uploaded to the Hadoop Distributed File System using the hdfs dfs -put command. HDFS handles replication and block distribution automatically, providing fault-tolerant storage.

Stage 3 — MapReduce Processing: Six MapReduce jobs are executed sequentially via Hadoop Streaming with Python mappers and reducers. Each job reads from the same input file in HDFS and writes its results to a separate output directory. The Hadoop framework handles the shuffle and sort phase between map and reduce, which is critical for grouping records by composite keys.

Stage 4 — Visualization: Results from all six jobs are downloaded from HDFS, converted to JSON format, and fed into an interactive web dashboard built with Chart.js and deployed on Vercel. The dashboard provides six analytical views corresponding to the six MapReduce jobs.

3.2 MapReduce Job Design

The core of the system consists of six MapReduce jobs, each designed to answer a specific analytical question. The key design decision in each job is the choice of composite keys and the aggregation logic in the reducer.

Job	Analysis	Map Output Key	Reduce Operation
1	Export Volume by Country	partner_country	SUM(trade_value)
2	Top Products per Market	country | HS_code | description	SUM(trade_value), RANK
3	Price Trend Analysis	HS_code | description | year	AVG, MIN, MAX(unit_price)
4	Emerging Market Growth	country | year	SUM per year, growth %
5	Volume by Country by Year	country | year	SUM(trade_value)
6	Top Products by Year	year | HS_code | description	SUM(trade_value), RANK

Jobs 1 and 2 provide the foundational country-level and product-level rankings. Job 3 introduces a derived metric (unit price = trade_value / net_weight), requiring additional validation to filter out records with zero weight. Jobs 4, 5, and 6 add the temporal dimension, enabling year-over-year analysis. The composite key design using pipe-delimited fields (e.g., country|year) leverages Hadoop's default lexicographic sort to ensure proper grouping in the reduce phase.

4. Platform and Tools

The following technologies were used in the implementation of this project:

Technology	Role in the Project
Apache Hadoop 3.3.6	Core distributed computing framework. Provides HDFS for fault-tolerant storage (with automatic block replication) and YARN for resource management and job scheduling across the cluster.
HDFS	Stores the 96.4 MB cleaned dataset with built-in replication. Files are split into 128 MB blocks (our file fits in a single block) and distributed across DataNodes.
Hadoop Streaming	Enables the use of Python scripts as mappers and reducers instead of Java. Data flows through stdin/stdout, making development and debugging faster than native Java MapReduce.
Python 3	Used for all processing scripts: data acquisition from UN Comtrade API, data cleaning and validation, MapReduce mapper/reducer logic, and JSON post-processing.
UN Comtrade API	Official United Nations trade statistics database. Provides programmatic access to real trade records classified by the Harmonized System (HS) commodity codes.
Chart.js 4.4.1	Client-side JavaScript charting library used in the dashboard for bar charts, line charts, area charts, and doughnut charts.
Vercel	Cloud deployment platform hosting the interactive dashboard at https://uncomtrade-bigdata.vercel.app/
Oracle VirtualBox	Virtualization platform running Ubuntu Linux as the development and execution environment for the Hadoop pseudo-distributed cluster.

We chose Hadoop Streaming with Python over native Java MapReduce for several reasons. Python's csv module handles quoted CSV fields natively, which was essential for our dataset where product descriptions contain commas. Python also enabled rapid prototyping: the same mapper/reducer logic could be tested locally with Unix pipes (cat input | python mapper.py | sort | python reducer.py) before deploying to Hadoop, significantly reducing the development cycle.

The system runs on a pseudo-distributed Hadoop cluster (single node acting as both NameNode and DataNode). While this configuration does not demonstrate horizontal scaling across physical machines, the MapReduce jobs, HDFS storage, and YARN resource management are identical to a production multi-node deployment. The code would run without modification on a cluster of any size.

5. Implementation

5.1 Data Acquisition

Trade data was obtained from the UN Comtrade API (comtrade - v1) using a free subscription key. A Python script was developed to automate the download process, fetching 100,000 records per API call for each year from 2018 to 2023. The initial download encountered HTTP 500 errors for years 2021 and 2022 due to server-side issues, resulting in only 400,000 records. A retry mechanism was implemented, and the missing years were successfully recovered, bringing the total to 600,000 records (97.5 MB).

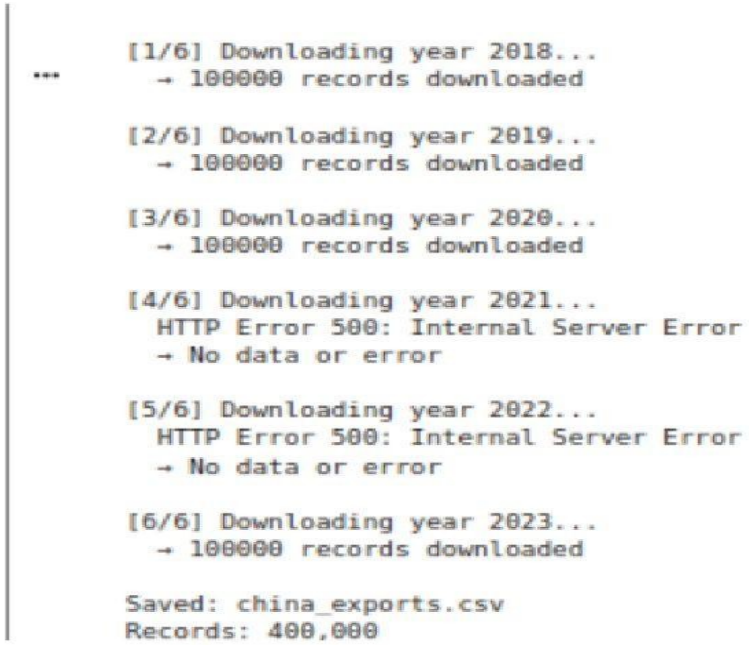


Figure 1: Initial API download showing HTTP 500 errors for 2021 and 2022

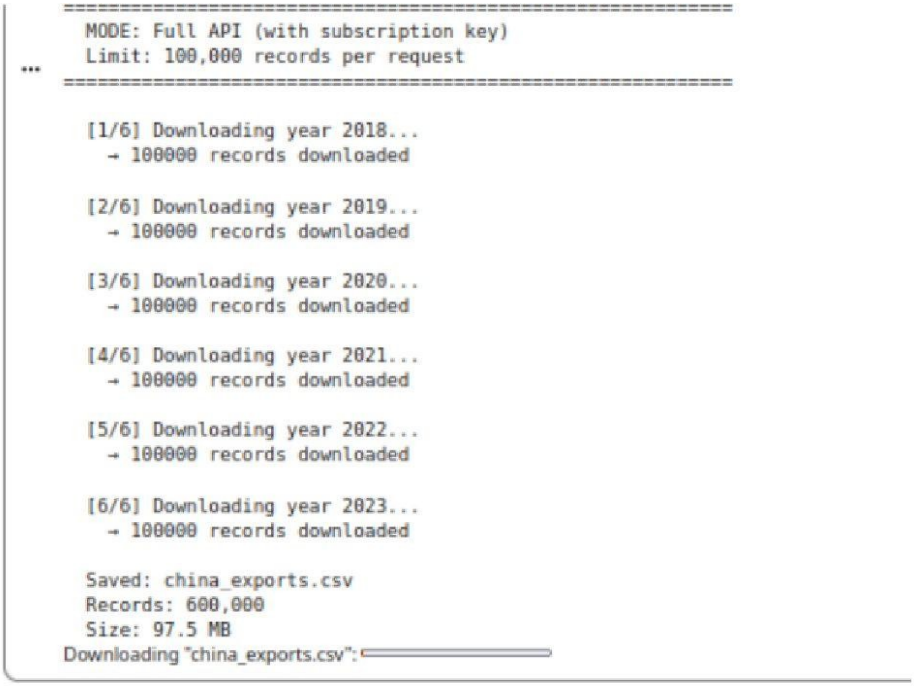


Figure 2: Successful download of all 600,000 records after implementing retry logic

5.2 Data Cleaning

A comprehensive data cleaning script was developed to preprocess the raw CSV before HDFS ingestion. The cleaning operations include: removal of duplicate headers introduced by the append-based download process, filtering of aggregate entries such as "World" and "Areas, nes" that would skew country-level analysis, validation of numeric fields (trade value, net weight, quantity), normalization of country names for consistency (e.g., "Viet Nam" to "Vietnam", "Korea, Republic of" to "South Korea"), and removal of exact duplicate rows.

The cleaning process removed 14,014 rows (2.3% of the dataset), retaining 97.7% of the original data. The majority of removed records were aggregate partner entries. The cleaned file size was reduced from 97.5 MB to 96.4 MB.

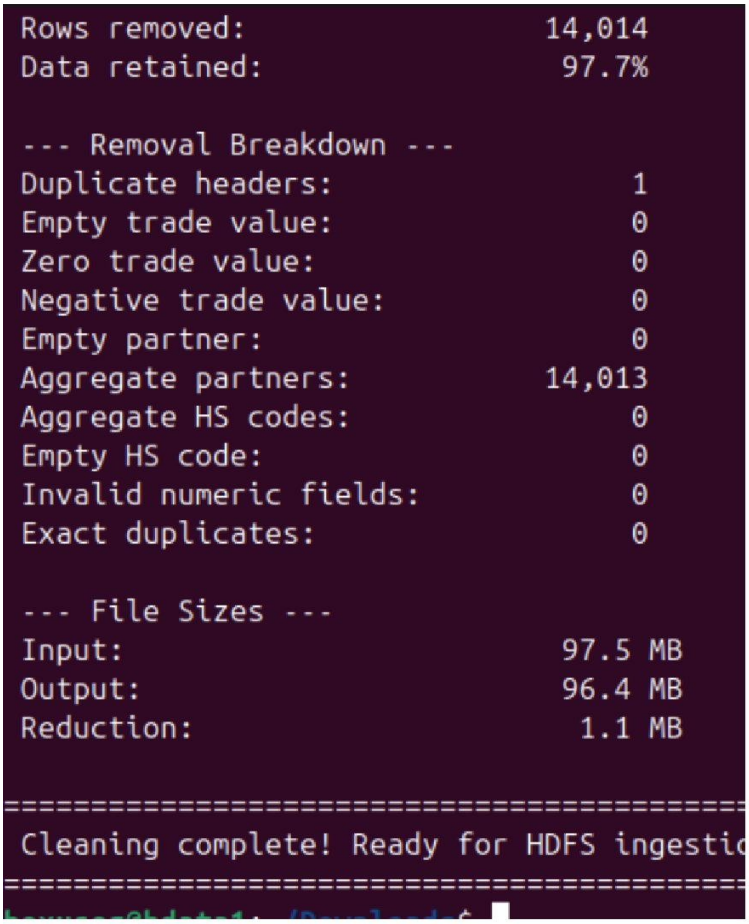


Figure 3: Data cleaning report showing removal breakdown and file size reduction

5.3 Hadoop Cluster Setup and HDFS Ingestion

The Hadoop cluster was started using start-dfs.sh and start-yarn.sh scripts. Cluster health was verified using the jps command, confirming all six required Java processes: NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager, and Jps.

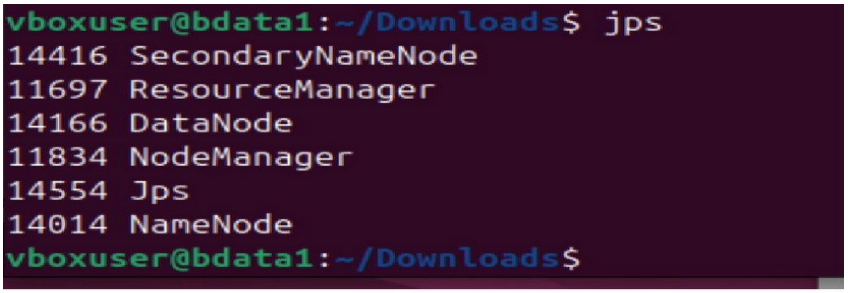


Figure 4: JPS output confirming all six Hadoop daemons are running

The cleaned dataset was uploaded to HDFS and verified:

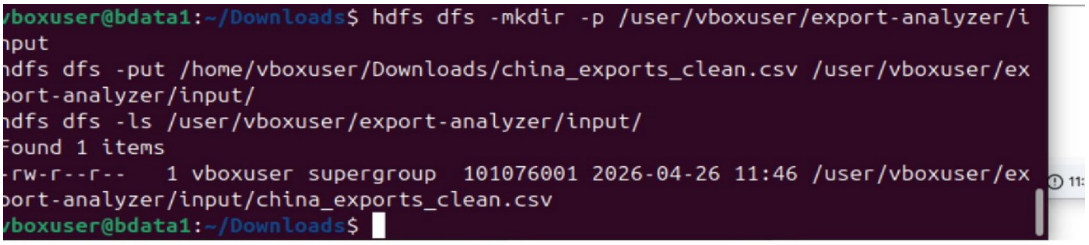


Figure 5: HDFS directory creation, file upload, and verification showing the 101 MB file

5.4 MapReduce Job Execution

Each MapReduce job was executed using the hadoop jar command with the Hadoop Streaming JAR. The mapper and reducer Python scripts were distributed to the cluster nodes via the -files parameter. Below is the execution pattern for Job 1 as a representative example:

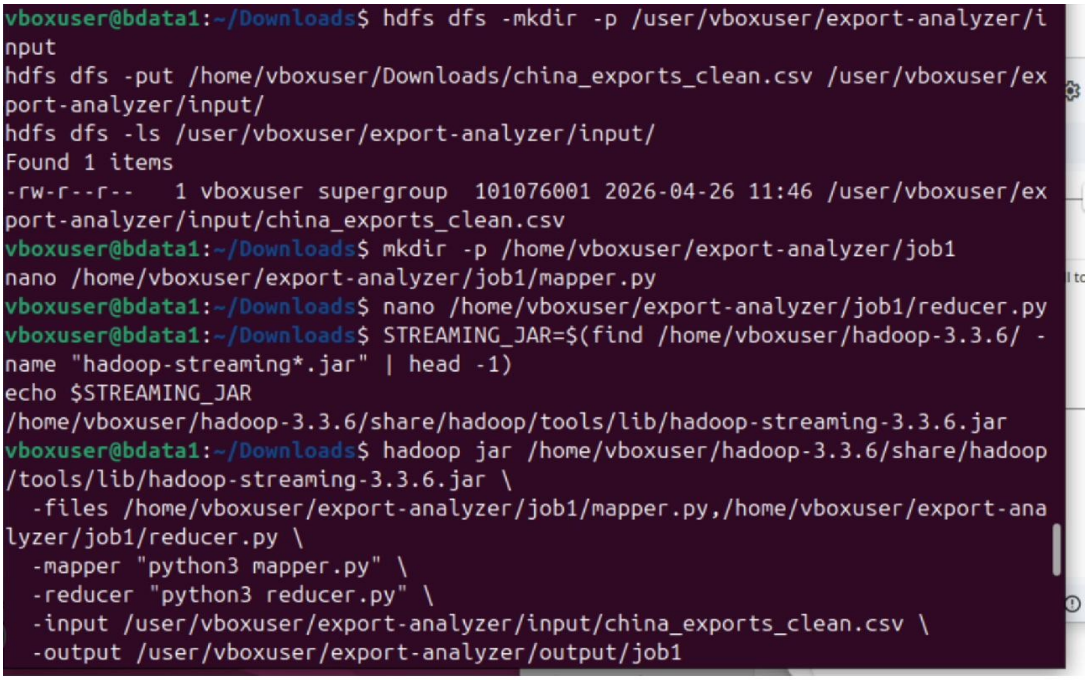


Figure 6: Complete workflow: HDFS operations, Streaming JAR discovery, and Job 1 execution

The mapper code for Job 1 demonstrates the pattern used across all jobs: reading CSV records via Python's csv.reader for proper handling of quoted fields, extracting relevant fields, applying filters, and emitting key-value pairs separated by a tab character:

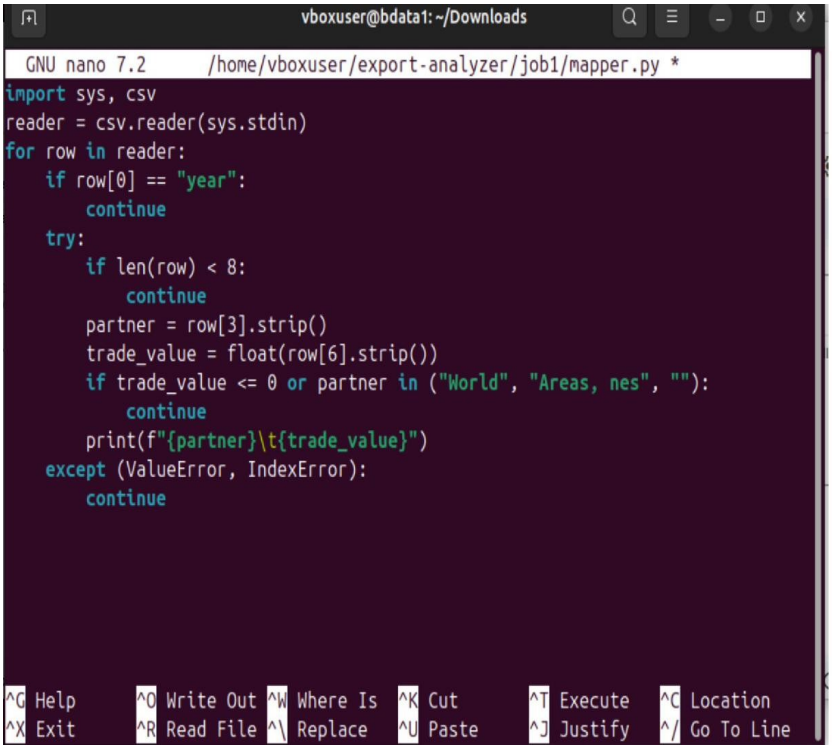


Figure 7: Job 1 mapper.py: reads CSV, filters invalid records, emits (country, trade_value) pairs

6. Results and Demonstration

6.1 Job 1: Export Volume by Country

The first MapReduce job produced a complete ranking of 216 destination countries by total export value across 2018-2023. The results confirm that the United States is China's largest export market at $1.14 trillion, followed by Hong Kong ($469B), Japan ($289B), South Korea ($242B), and Vietnam ($194B).

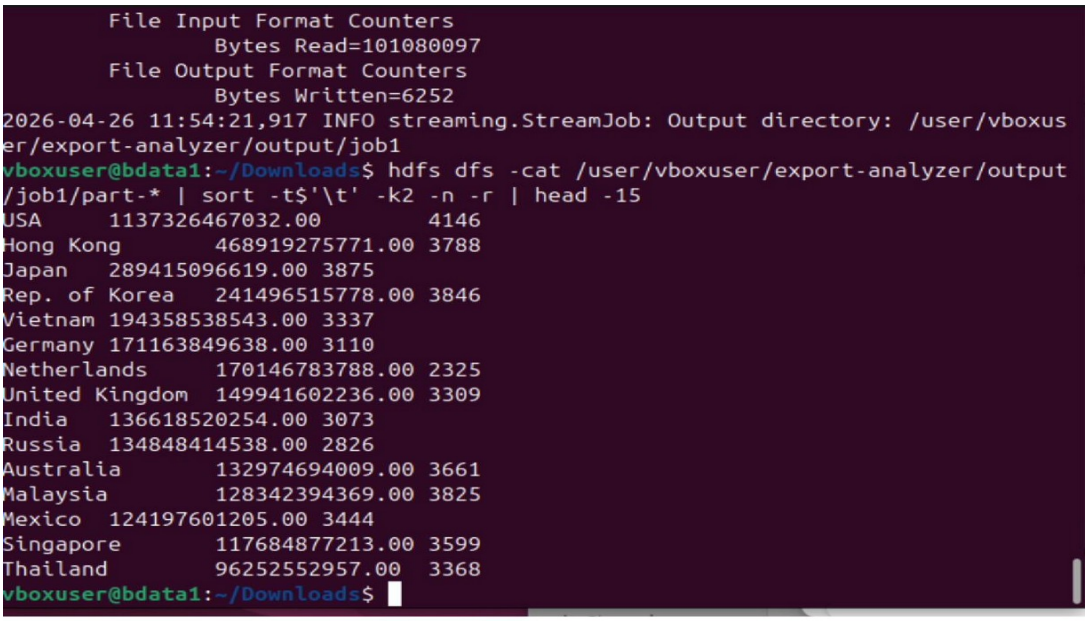


Figure 8: Job 1 output: Top 15 export destinations ranked by total trade value (USD)

6.2 Job 2: Top Products per Market

Job 2 revealed the product composition of each market. For the United States, data processing machines (HS 8471) dominate at $154.2B, followed by portable computers at $128.2B and smartphones at $80.3B. Notably, different markets show distinct product profiles: South Korea's top import from China is memory chips ($39B), while Russia's is motor vehicles ($14.9B), reflecting each economy's industrial structure.



Figure 9: Job 2 output: Top 10 export product categories for the USA market

6.3 Job 4: Emerging Markets

Job 4 identified countries with the highest export growth rates. Among significant markets (those with initial volumes exceeding $500M), Kyrgyzstan showed a remarkable +1,239% growth from 2020 to 2021, followed by France (+919%) and the Netherlands (+818%). These figures largely reflect the post-COVID recovery and supply chain restructuring that occurred in 2021.

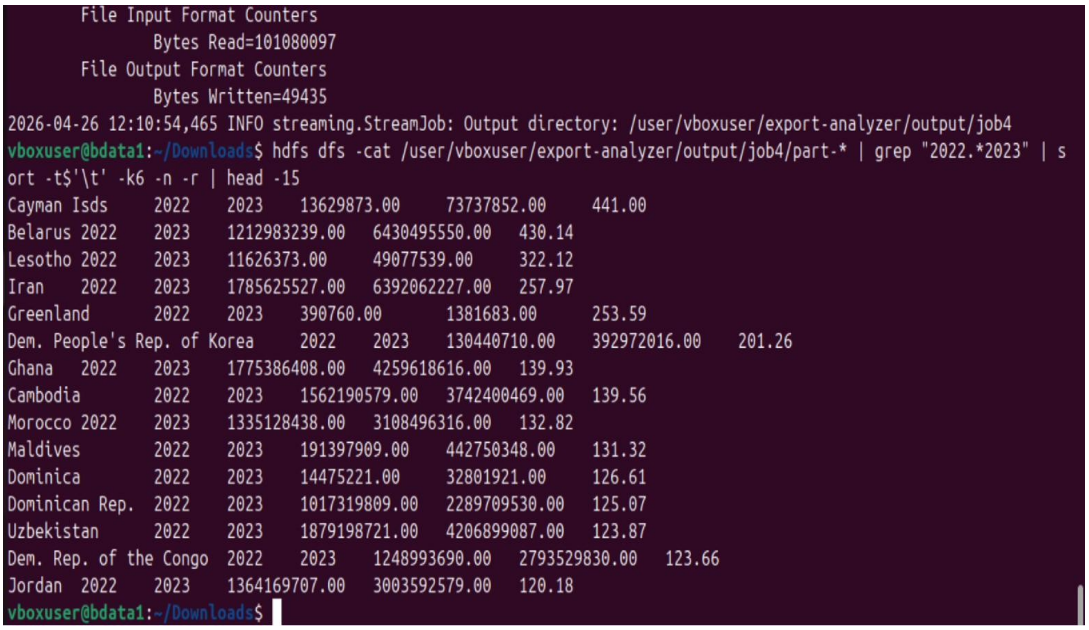


Figure 10: Job 4 output: Countries with highest year-over-year export growth rates

6.4 Job 5: Yearly Evolution

Job 5 enabled tracking individual country trends over time. The data for the United States reveals a dramatic arc: exports grew from $21.6B in 2018 to a peak of $386.9B in 2022, before declining to $213.3B in 2023. This pattern reflects both the post-pandemic demand surge for electronics and the subsequent normalization of trade volumes.

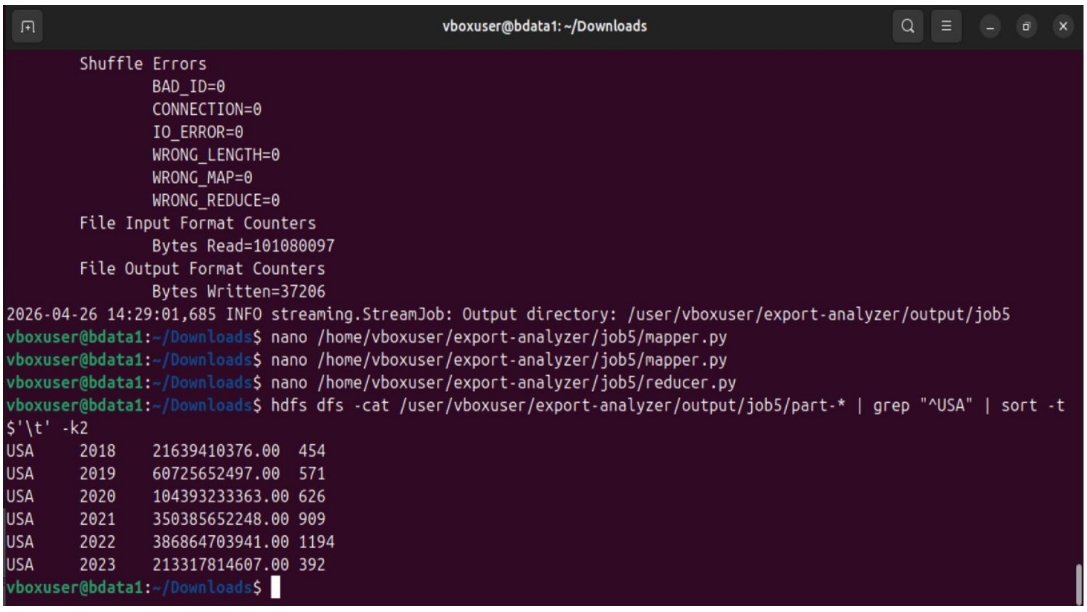


Figure 11: Job 5 output: USA export volume evolution from 2018 to 2023

6.5 Interactive Dashboard

All MapReduce results were integrated into an interactive web dashboard deployed at https://uncomtrade-bigdata.vercel.app/. The dashboard provides six analytical views, each corresponding to one MapReduce job. Users can switch between views, select specific countries or years, and hover over data points for detailed tooltips.



Figure 12: Dashboard Overview: Top Export Destinations bar chart with key statistics

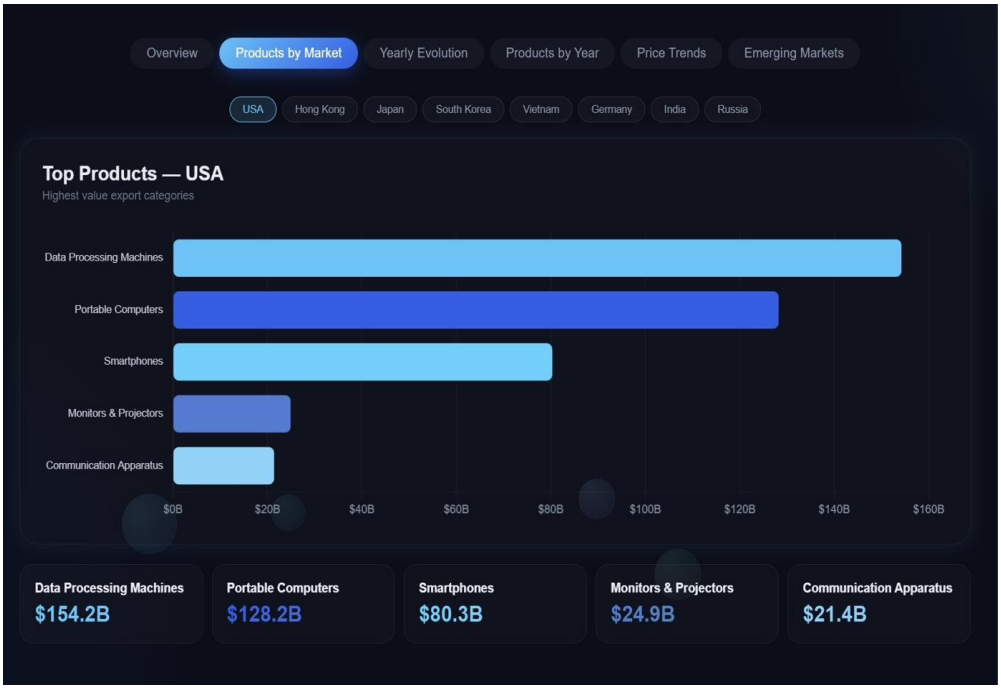


Figure 13: Dashboard Products by Market: interactive country selector with USA products

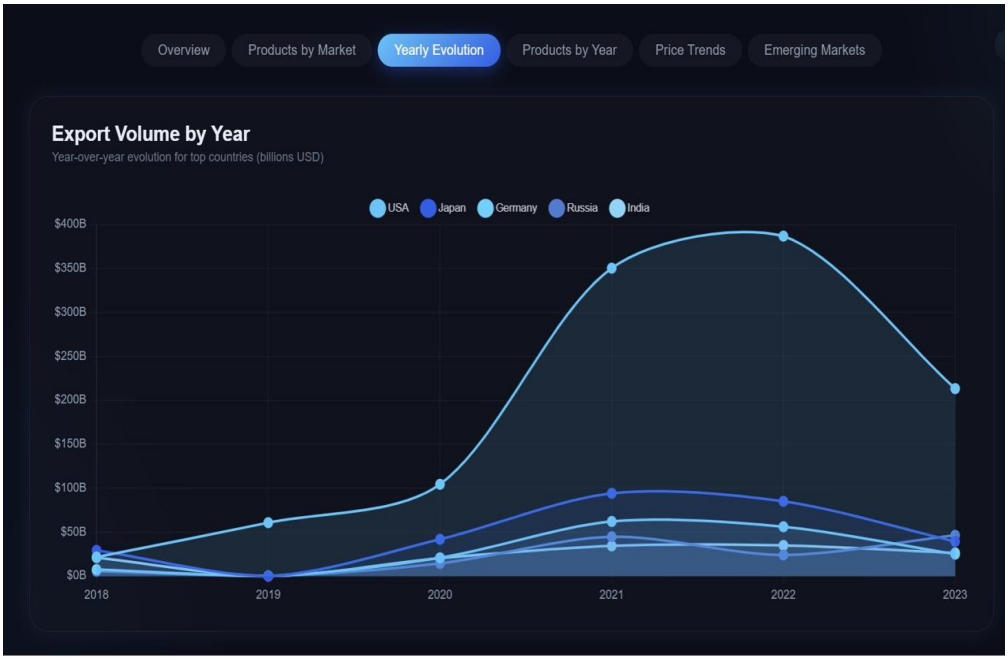


Figure 14: Dashboard Yearly Evolution: multi-country area chart showing trade volume trends

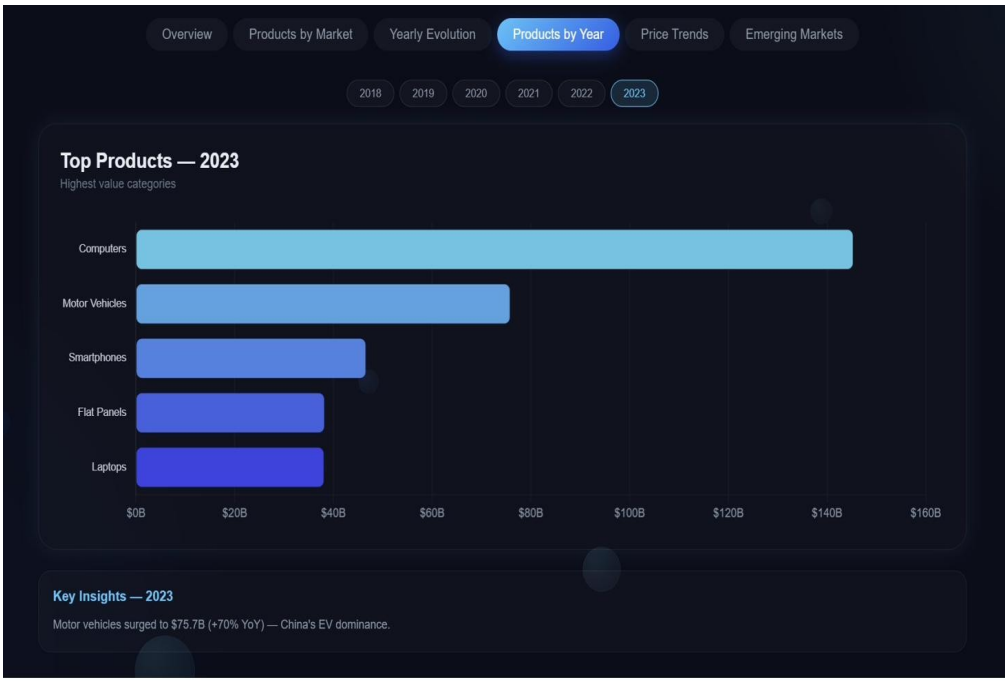


Figure 15: Dashboard Products by Year: 2023 view showing the EV surge with contextual insights

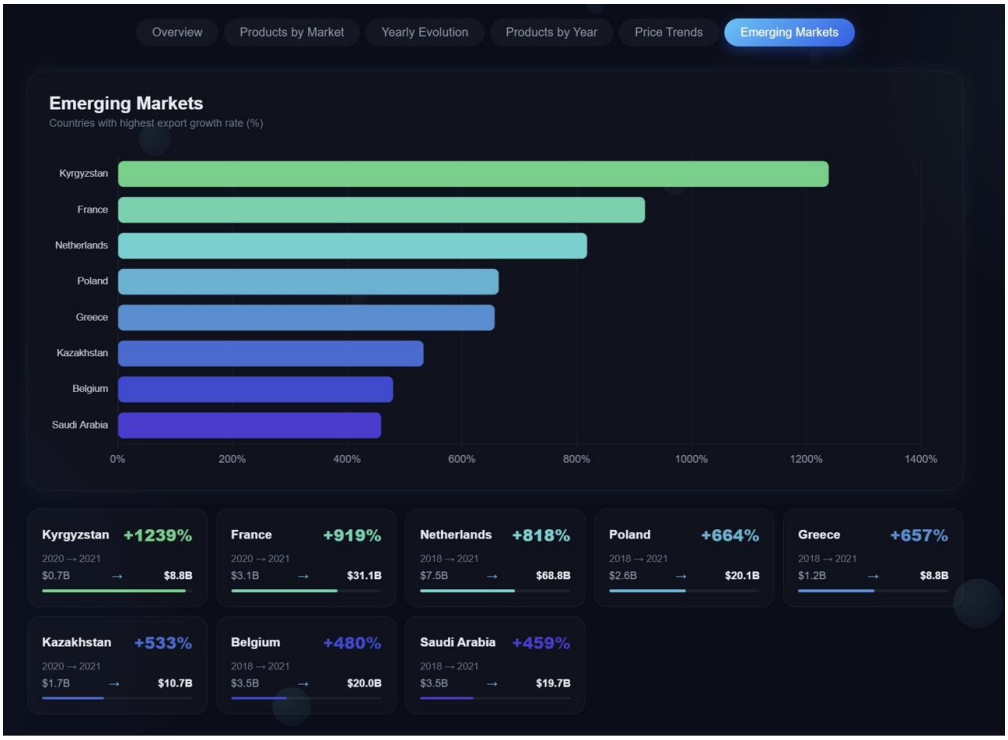


Figure 16: Dashboard Emerging Markets: growth rate chart with detailed country cards

7. Difficulties and Solutions

7.1 NameNode Failure on Startup

Upon initial startup of the Hadoop cluster, the jps command revealed that only five of six required daemons were running. The NameNode, which is the master daemon responsible for managing the HDFS namespace, failed to start. Attempting to access HDFS resulted in a "Connection refused" error on port 9000.

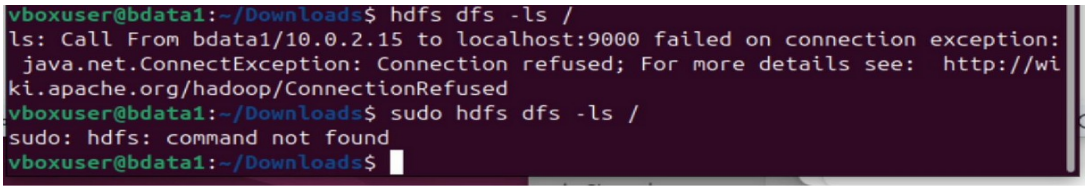


Figure 17: HDFS connection refused error indicating the NameNode was not running

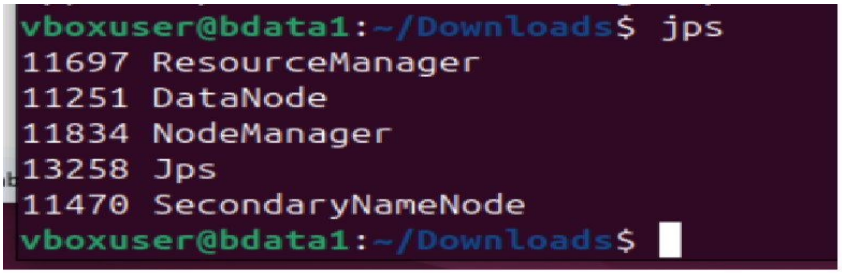


Figure 18: JPS showing only 5 processes — NameNode is missing

The solution was to format the NameNode filesystem using hdfs namenode -format and restart the DFS daemons. This is a common issue when the NameNode's metadata directory becomes corrupted, particularly after abrupt shutdowns of VirtualBox saved states. The trade-off is that formatting erases all HDFS data, requiring re-upload of the dataset.

7.2 CSV Parsing with Embedded Commas

The UN Comtrade dataset contains product descriptions with embedded commas (e.g., "Woven fabrics of cotton, containing 85% or more by weight of cotton, weighing more than 200g/m2"). The initial mapper implementation used a simple string split on commas, which broke the field alignment and produced incorrect results.

The solution was to replace the naive split with Python's csv.reader module, which properly handles quoted fields containing commas. This required reading from sys.stdin through csv.reader instead of processing lines directly, a small change in code but critical for correctness.

7.3 API Rate Limiting and Server Errors

During the initial data download from UN Comtrade, the API returned HTTP 500 (Internal Server Error) for years 2021 and 2022, resulting in only 400,000 out of the target 600,000 records. Rather than accepting the incomplete dataset, we implemented a retry mechanism with a 10-second delay between requests, which successfully recovered the missing 200,000 records. This experience underscores the importance of building resilient data pipelines that can handle transient failures in external data sources.

7.4 Dashboard CDN Compatibility

The initial dashboard implementation used React and Recharts loaded via CDN with Babel for JSX transpilation. When deployed to Vercel, the page rendered as a blank screen because the

Babel CDN script failed to load consistently. The solution was to rewrite the entire dashboard using plain JavaScript (no JSX) with Chart.js, which is a standalone library with no framework dependencies. This eliminated the complex dependency chain and produced a reliable, single-file deployment.

8. Conclusion

8.1 Summary of Achievements

This project successfully designed and implemented a complete Big Data analytics pipeline for international trade data. The system processes 600,000 real trade records from the UN Comtrade database through six MapReduce jobs running on Apache Hadoop, producing actionable insights about China's export patterns across 216 countries and 5,798 product categories over the period 2018-2023.

Key findings from the analysis include: the dominance of electronics (computers, smartphones, laptops) in China's export profile; the dramatic impact of COVID-19 on trade patterns, with textile/PPE exports surging to $42.6B in 2020; the rapid rise of electric vehicle exports from $44.6B in 2022 to $75.7B in 2023; and the identification of emerging markets such as Kyrgyzstan, France, and the Netherlands showing growth rates exceeding 800%.

8.2 Lessons Learned

The project provided hands-on experience with the complete lifecycle of a Big Data application: from data acquisition through a REST API, to distributed storage in HDFS, parallel processing with MapReduce, and interactive visualization. Key takeaways include the importance of data cleaning as a prerequisite for meaningful analysis, the power of the MapReduce paradigm for large-scale aggregations, and the practical challenges of working with real-world datasets that contain inconsistencies and missing values.

8.3 Future Work

Several directions for future improvement have been identified:

- Migration to Apache Spark for in-memory processing, which would significantly reduce execution time for iterative analyses and enable interactive querying.
- Integration of real-time streaming using Spark Streaming or Apache Kafka to process live trade data as it becomes available from customs agencies.
- Development of a more comprehensive web dashboard with user authentication, saved queries, and export functionality.
- Application of machine learning models for demand prediction, anomaly detection, and trade pattern forecasting.

References

[1] Apache Software Foundation. "Apache Hadoop." https://hadoop.apache.org/ (accessed April 2026).

[2] Apache Software Foundation. "Hadoop Streaming." https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html (accessed April 2026).

[3] Apache Software Foundation. "HDFS Architecture Guide." https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html (accessed April 2026).

[4] United Nations Statistics Division. "UN Comtrade Database." https://comtradeplus.un.org/ (accessed April 2026).

[5] United Nations Statistics Division. "UN Comtrade API Documentation." https://comtradedeveloper.un.org/ (accessed April 2026).

[6] Chart.js Contributors. "Chart.js: Simple yet flexible JavaScript charting." https://www.chartjs.org/ (accessed April 2026).

[7] World Customs Organization. "Harmonized System Nomenclature." https://www.wcoomd.org/en/topics/nomenclature/overview.aspx (accessed April 2026).

[8] Python Software Foundation. "csv — CSV File Reading and Writing." https://docs.python.org/3/library/csv.html (accessed April 2026).

Appendix

A. Source Code Repository

The complete source code is available on GitHub:

https://github.com/MaximoFox487/UNCOMTRADE-BIGDATA

B. Live Dashboard

The interactive dashboard is publicly accessible at:

https://uncomtrade-bigdata.vercel.app/

C. Project File Structure

export-analyzer/

data/clean_data.py — Data preprocessing and validation

data/download_comtrade.py — API data acquisition script

job1/mapper.py, reducer.py — Export Volume by Country

job2/mapper.py, reducer.py — Top Products per Market

job3/mapper.py, reducer.py — Price Trend Analysis

job4/mapper.py, reducer.py — Emerging Market Growth

job5/mapper.py, reducer.py — Volume by Country by Year

job6/mapper.py, reducer.py — Top Products by Year

process_results.py — JSON generation for dashboard

index.html — Interactive web dashboard